Abstain or Prove: Exact Recovery with Refusal as a First-Class Output

Jacob Iannotti
jiannotti5040@gmail.com

Draft of July 4, 2026

Abstract

Most inductive systems return their best hypothesis for every input; their failures are therefore confident. We describe an engine built around the opposite contract: recover the minimal generator beneath a codified surface under a two-part MDL criterion in exact rational arithmetic, stamp the result VERIFIED only when the recovered rule exactly predicts held-out data it never saw, and otherwise *refuse*. Refusal is a first-class output, not an error. We report three kinds of evidence. (1) **External validation:** on 25 sequences fetched live from the OEIS, the engine verifies 18 — every one externally correct — and refuses 6, with zero false stamps; the refusals fall exactly where its documented hypothesis classes end (primes, partitions, Bell, Stern, Thue–Morse, divisor and totient functions). (2) **Falsification as method:** the first external run caught a false verification that ∼5,000 internally generated cases had never surfaced (float drift in a recurrence path passing a relative tolerance); we describe the repair — a systematic purge of floating point from every stamping path — and the differential-testing and fuzzing infrastructure that now makes silent regression a build failure, which promptly caught two further defects before release. (3) **Contract comparison:** against a genetic-programming symbolic-regression baseline under an identical protocol, the engine is 16-exact / 0-wrong / 8-refused where the regressor is 2-exact / 22-wrong — not because it fits better, but because a regressor cannot say "I don't know." We argue calibrated refusal, not recovery breadth, is the property that matters when machine outputs feed decisions, and we ship the discipline as an installable package and MCP tool that gates language-model output claim by claim.

1 Introduction

A system that must always answer converts every gap in its competence into a confident error. This is tolerable when outputs are suggestions; it is not tolerable when outputs gate decisions. We take the position that for machine inference to deserve influence, *the refusal must be engineered as carefully as the answer.*

The engine described here (PRIMUS, the auditable seed of a larger system, CHIRON) implements a four-step contract: *recover* the minimal generator beneath a codified surface; *verify* it by exact prediction of withheld data; *refuse* otherwise; *certify* the outcome in an auditable record. The defining property is not recall. It is that in every evaluation we have run — internal, external, and adversarial — the engine has never certified a rule it could not exactly predict with, and when it was once caught doing so (Section 4), the defect was repaired at its root rather than tolerated.

Contributions: (i) an exact-arithmetic recovery engine over an explicit hypothesis lattice with held-out verification and refusal (Section 3); (ii) an externally validated zero-false-stamp result on live OEIS data, including exact P-recursive recovery one order beyond rational ratios (Section 5); (iii) a methodology — external falsification, float purging, differential testing with a capability ledger, adversarial fuzzing — that treats the verifier itself as the object under test (Section 4); (iv) a certificate layer that applies the same discipline to language-model output, with an honest account of its blind spot (Section 7).

2 Related work

MDL and induction. Two-part MDL model selection follows Rissanen [1]; the idealized limit is Solomonoff induction [2]. Our departure is not the criterion but the contract wrapped around it: exact holdout or refusal. **Sequence recognition.** The OEIS Superseeker [3] and holonomic guessing tools (`gfun` [4], Kauers's `Guess` [5]) recover the same families we target; they are lookup or fitting services and do not expose a calibrated refuse-vs-certify boundary as their output semantics. **Symbolic regression.** Genetic-programming SR [6, 7] and modern systems such as PySR [8] and AI Feynman [9] return best-fit expressions; abstention is not native to the interface (Section 6). **Selective prediction.** Rejection and selective classification go back to Chow [10]; see El-Yaniv and Wiener [11] and conformal prediction [12]. These bound *error rates* statistically; our setting demands exactness, so the analogous guarantee is structural: a stamp exists only alongside an exact held-out reproduction. **Verifying LLM output.** Work on hallucination detection and process verification typically scores plausibility with another model. Our certificate layer is complementary and deliberately narrower: it checks only what is exactly checkable and refuses the rest (Section 7).

3 The contract

3.1 Surfaces and hypothesis classes

A *surface* is any codified object: an integer sequence, a string, a ciphertext, a graph, a schema, source code. For numeric surfaces the engine searches an explicit lattice: constant; polynomial (recovered on integer surfaces by exact Newton finite differences); geometric (exact rational ratio); constant-coefficient linear recurrences of order ≤ 4 (coefficients snapped to exact rationals and validated by exact regeneration); P-recursive (holonomic) recurrences $\sum_{j=0}^{r} p_j(n)\, a(n+j) = 0$ with integer polynomial coefficients, $r \leq 2$, $\deg p_j \leq 2$, found by exact Fraction nullspace elimination; periodic, alternating, multiplicative-ratio, and power-law families. The class boundary is documented, and everything outside it is a refusal by design, not an aspiration.

3.2 Selection and verification

Among candidates that exactly reproduce (or, for float surfaces, compress) the shown terms, two-part MDL selects the winner, with a canonical tie-break so that, e.g., a line reads as *arithmetic* and a polynomial multiple of a constant-coefficient recurrence is excluded from the holonomic family (the rank-one exclusion; without it Fibonacci can be dressed up as holonomic — caught by our stress suite, Section 4). Verification then refits on a prefix and demands the refit rule predict the held-out tail *at exact integer equality* on integer surfaces; floating predictions above

2^{53} are refused rather than trusted. VERIFIED means precisely this and nothing more: the rule reproduced data it had never seen, exactly.

4 Falsification as method

4.1 The repunit episode

The engine's internal benchmark (110 sequences, 98% recovery, 100% precision-of-stamp) and ∼5,070-case fuzz gauntlet showed zero false verifications. The first *external* run promptly falsified that record: repunits (OEIS A002275), a true order-2 recurrence $a(n) = 11a(n-1) - 10a(n-2)$, were stamped VERIFIED while predicting 111111112740 where the OEIS says 111111111111. Two compounding defects: least-squares recurrence coefficients carried $\sim 10^{-8}$ relative float error that compounds geometrically under iteration; and the holdout comparison used a 10^{-6} *relative* tolerance — at $a(n) \approx 10^{10}$ it forgave absolute errors of $\sim 10^4$ — despite documentation promising exact equality.

The repair was structural, not local: every stamping path was purged of floating point. Recurrence coefficients snap to exact rationals and must regenerate every shown term in Fraction arithmetic; polynomials on integer surfaces are recovered by exact finite differences (predictions are exact integers, valid far beyond 2^{53}); geometric recovery uses exact ratio detection (as a corollary, negative ratios — structurally invisible to log-space regression — became verifiable); P-recursive recovery was built exact from the start. The production system's consolidated engine had independently evolved past the defect while the "auditable seed" silently retained it — a concrete demonstration of multi-copy drift.

4.2 Making regression structural

Three mechanisms now stand between the engine and silent regression. **Differential testing:** a 34-surface battery runs through both the seed and the consolidated engine on every build; if both stamp with different predictions, or the seed stamps where the monolith abstains without a dated *capability-ledger* entry, the build fails. The ledger was exercised in earnest the same day it was written: the exact P-recursive solver made the seed verify Motzkin while the consolidated engine — whose own holonomic solver was already exact, but demanded an overdetermination margin unformable on holdout prefixes — abstained. The divergence failed the build, received a written reason, and was cleared within hours by porting the margin fix; the ledger is empty again, and both engines verify Motzkin and Schröder identically. **Adversarial fuzzing:** thirteen gates throw hostile input at the certificate layer; before first release they found a denial-of-service (a 20,000-integer flood becoming a single unbounded recovery call) and a quadratic scanning cost, both fixed with deterministic work bounds. **External re-validation:** the OEIS harness is a required gate in continuous integration, pinned to a live-fetched corpus and re-runnable against the live database at any time.

5 External validation on live OEIS data

Protocol. 25 sequences fetched live from `oeis.org` (per-sequence b-files; selection rule fixed before any sequence was run: canonical sequences spanning the declared classes *plus* famously non-closed-form sequences where refusal is correct, plus boundary probes). The engine sees the first 12 terms; the recovered rule must predict terms 13–16 exactly against OEIS ground truth.

Grade	n	Sequences
VERIFIED + externally correct	18	Fibonacci, Lucas, Pell, Tribonacci, Jacobsthal, 2^n, 3^n, squares, cubes, triangular, oblong, factorials, repunits, Catalan, central binomial, quarter-squares, **Motzkin**, **Schröder**
VERIFIED + wrong (false stamp)	**0**	—
recovered, conservatively unstamped	1	Thue–Morse
REFUSED (correct refusals)	6	primes, partitions, Bell, $d(n)$, $\varphi(n)$, Stern

Table 1: Live-OEIS battery, 12-shown / 4-graded, exact equality. Reproduce with `python3 Primus/oeis_live.py`.

Two details deserve emphasis. First, **Motzkin** was this battery's identified miss (order-2 P-recurrence, one step beyond rational ratios); the exact nullspace solver recovers its classical recurrence $(n{+}4)M(n{+}2) = (2n{+}5)M(n{+}1) + 3(n{+}1)M(n)$ from 12 terms and verifies on held-out values. **Schröder** was then fetched *after* the solver existed, as a fresh out-of-development probe, and verified on first contact. Second, **Bell numbers still refuse** — they are not P-recursive — and the control holding is as load-bearing as the recoveries: a solver that had merely memorized more sequences would not know where to stop.

What this does not show. The recovered rules are rediscoveries of classified structure, not new mathematics; the battery is curated, not the full OEIS. The harness supports the escalation (`oeis_live.py --live --keyword-core` over the ∼180-sequence core corpus, and onward to the full database); **TODO(author): run and report the full-corpus sweep.**

6 Contract comparison with symbolic regression

Under the identical protocol (12 shown, 4 graded exactly), a genetic-programming baseline (gplearn, population 300, 12 generations) scores 2-exact / 22-wrong; the engine scores 16-exact / 0-wrong / 8-refused (this run predates the P-recursive solver; today's engine adds Motzkin and Schröder). The point is deliberately *not* that GP fits badly — exact integer continuation is a hostile bar for it, and larger budgets or stronger systems (PySR) recover more closed forms. **TODO(author): run `bench_pysr.py` (requires Julia) and report the same table for PySR.** The structural point survives any budget: on sequences with no recoverable closed form, the best possible regressor output is a confident wrong answer, because abstention is not in the interface. The property under test is calibrated confidence, and it is the one a regressor cannot buy with compute.

7 The certificate layer

The same discipline wraps language-model output. CERTIFY extracts the *checkable* claims from text — exact arithmetic (including powers), percentages, primality (deterministic Miller–Rabin below the proven witness bound, REFUSED above it), binomial coefficients, gcd/lcm, modular and calendar-exact date arithmetic, sums and averages of listed numbers, and integer-sequence continuations checked through the engine — and stamps each VERIFIED, REFUTED, or REFUSED. Free text is reported as unverifiable, never blessed.

The gate semantics are stated in the schema and repeated here because they are the layer's most important sentence: *a passing gate means nothing checkable was refuted; it never means the output is true.* A claim phrased outside the extractor's patterns is not caught — a blind spot structural to any extractor, which this one mitigates by reporting `coverage` (the fraction of input actually checked) so a pass at 2% coverage cannot masquerade as one at 80%. All scanning is bounded against hostile input (truncation, claim caps, integer-size caps, per-claim sequence caps, and anchor-windowed matching), and the certificate's hash is labeled what it is — tamper-evident, not an unforgeable signature. The layer ships as a Python package, CLI gate (exit nonzero on any REFUTED claim), and a dependency-free MCP server, so agents can call the verifier natively.

8 Limitations

The hypothesis lattice is explicit and therefore finite; everything outside it refuses, including structures a broader system would recover. The external battery is 25 curated sequences; the full-corpus sweep is pending. The certificate layer's recall over real model output is bounded by its extractor. Verification is exact and therefore restricted to discrete / rational structure; noisy float surfaces fall back to a weaker, clearly labeled path. All validation to date is by the author and one automated collaborator; independent replication is the point of shipping the harnesses. **TODO(author): consider a machine-checked (e.g. Lean) proof of the holdout gate's soundness on the integer path.**

9 Conclusion

Zero false verifications is not a benchmark score; it is a contract, and contracts are only credible after they have been attacked. This one was falsified once from outside, repaired at the root, and has since held under external data, differential testing, and adversarial fuzzing — with every refusal landing where the documentation said it would. We think this inversion — engineering the refusal as carefully as the answer — is the missing primitive for machine outputs that feed decisions, and we have tried to ship it in a form small enough to audit and easy enough to call.

References

[1] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[2] R. Solomonoff. A formal theory of inductive inference. *Information and Control*, 7(1–2), 1964.

[3] N. J. A. Sloane. The On-Line Encyclopedia of Integer Sequences. `https://oeis.org`.

[4] B. Salvy and P. Zimmermann. GFUN: a Maple package for the manipulation of generating and holonomic functions in one variable. *ACM TOMS*, 20(2):163–177, 1994.

[5] M. Kauers. Guessing handbook. RISC Report 09-07, 2009.

[6] J. Koza. *Genetic Programming*. MIT Press, 1992.

[7] T. Stephens. gplearn: genetic programming in Python. `https://github.com/trevorstephens/gplearn`.

[8] M. Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv:2305.01582*, 2023.

[9] S.-M. Udrescu and M. Tegmark. AI Feynman: a physics-inspired method for symbolic regression. *Science Advances*, 6(16), 2020.

[10] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Trans. Information Theory*, 16(1):41–46, 1970.

[11] R. El-Yaniv and Y. Wiener. On the foundations of noise-free selective classification. *JMLR*, 11:1605–1641, 2010.

[12] V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.